

avito.tech 

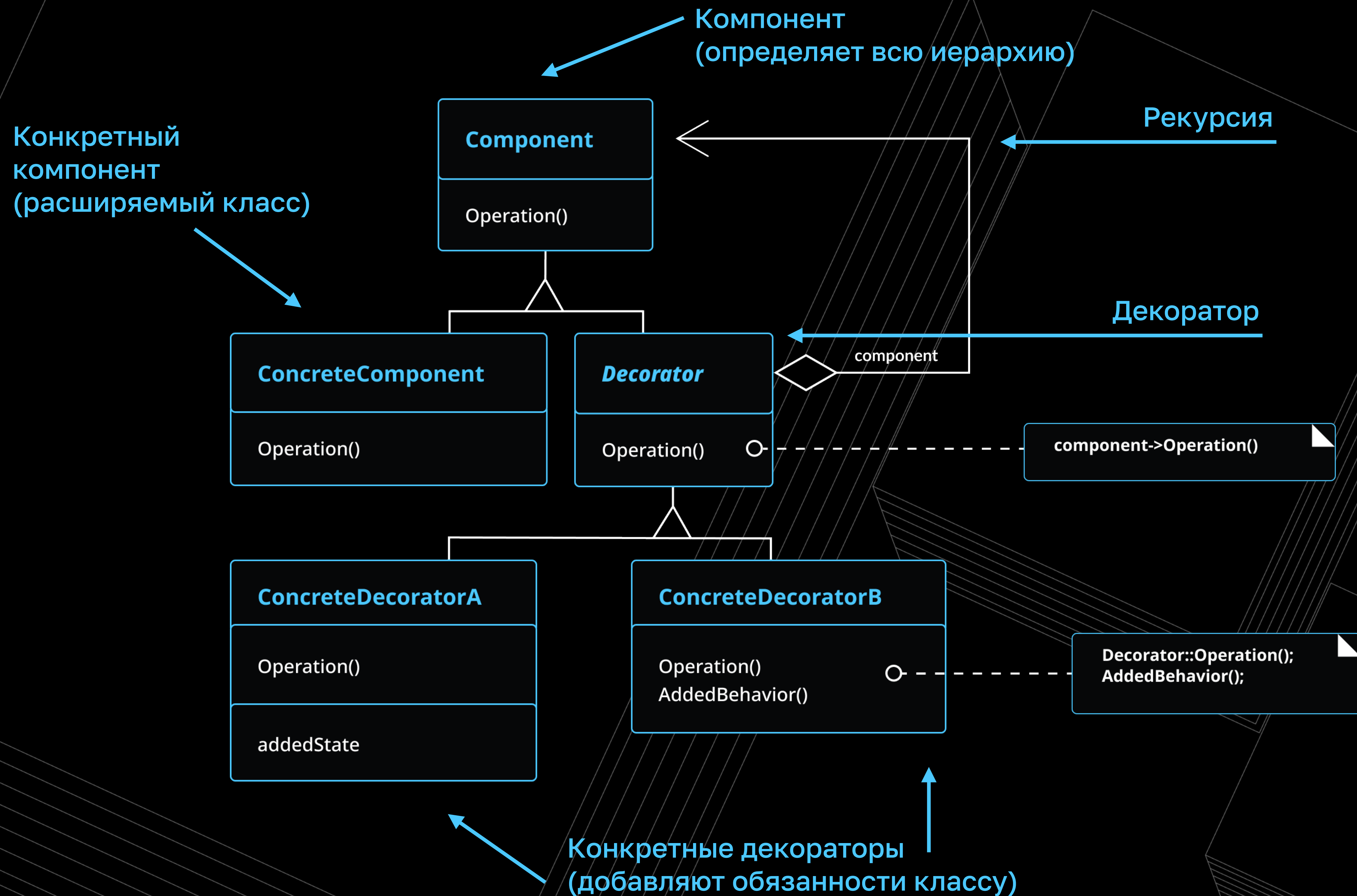
DECORATOR ДЕКОРАТОР

Афанасьев Юра

Динамически добавляет объекту
новые обязанности

Способы расширения класса

- Наследование
(статическая связь)
- Композиция или агрегация
(динамическая связь)
- Декоратор
(динамическая связь)



Конкретный
компонент

Компонент
(определяет всю
иерархию)

```
class ConcreteComponent implements Component
{
    public function operation(): string
    {
        return 'Result concrete component.' . PHP_EOL;
    }
}

// class AnotherConcreteComponent implements Component
// {
// }
```

На результат
метода будут
наложены
декораторы

Расширяемых классов
(компонентов)
может быть много

Компонент

Определяет интерфейс
для объектов
на которые возлагаются
дополнительные
обязанности

```
interface Component
{
    public function operation(): string;
}
```

Декоратор
(базовый для всех
декораторов)



```
abstract class Decorator implements Component
{
    protected Component $component;

    public function __construct(Component $component)
    {
        $this->component = $component;
    }
}
```

Конкретные
декораторы
(один класс
выполняет
одно действие/
поведение)

```
class ConcreteDecoratorA extends Decorator
{
    public function operation(): string
    {
        return $this->component->operation() . 'Result decorator A.' . PHP_EOL;
    }
}

class ConcreteDecoratorB extends Decorator
{
    public function operation(): string
    {
        return $this->component->operation() . 'Result decorator B.' . PHP_EOL;
    }
}

// class ConcreteDecoratorC extends Decorator
// {
// }
```

Итеративно вызываем
следующий метод
в цепочке

Вложение
декораторов
друг в друга

```
function testDecorator()  
{  
  echo (new ConcreteDecoratorC(  
    new ConcreteDecoratorB(  
      new ConcreteDecoratorA(  
        new ConcreteComponent()  
      )  
    )  
  ))->operation();  
}
```

Порядок
вкладывания
влияет на результат

Result concrete
component.
Result decorator A.
Result decorator B.
Result decorator C.

Вкладывание
можно реализовать
динамически

Плюсы

- Расширяет поведение и обязанности объекта во время выполнения программы
- Гибкая альтернатива наследованию
- Из большого количества классов получаем объект с необходимым набором функциональности
- Реализует принцип открытости/закрытости

Плюсы

- Возможность вкладываться друг в друга и образовывать любое число обязанностей
- Одну и ту же обязанность можно выполнить несколько раз

Выполнение
декоратора «В»
несколько раз

```
echo (new ConcreteDecoratorD(  
    new ConcreteDecoratorB(  
        new ConcreteDecoratorC(  
            new ConcreteDecoratorB(  
                new ConcreteDecoratorA(  
                    new ConcreteComponent()  
                )  
            )  
        )  
    )  
))->operation();
```

Минусы

The background features several abstract geometric shapes in a light blue color. These include a large rectangle on the left, a series of smaller rectangles and circles arranged in a diagonal line towards the center, and a large circle in the bottom left corner. The shapes are simple outlines with no fill.

- Множество мелких объектов, в работе которых нужно разобраться, прежде чем начать их применять

Пример из Java

Открываем поток
на чтение

Для ускорения
кладём в память

Разархивирует
файл

Десериализуем
Java объекты

Получаем
данные

```
FileInputStream fis = new FileInputStream("/objects.gz");  
BufferedInputStream bis = new BufferedInputStream(fis);  
GzipInputStream gis = new GzipInputStream(bis);  
ObjectInputStream ois = new ObjectInputStream(gis);  
SomeObject someObject = (SomeObject) ois.readObject();
```

Конкретные
декораторы

Почему Декоратор - структурный паттерн?

- Не занимается взаимодействием объектов
- Его задача в динамическом расширении поведения и обязанностей

Динамически
добавляет
дополнительное
поведение
или обязанности
объекту

Decorator

- Расширяет метод объекта
- Изменяет только обязанности объекта, но не его интерфейс

Adapter

- Позволяет заменить метод
- Придает объекту совершенно новый интерфейс

Decorator

- Добавляет новые обязанности, агрегирование объектов не является его целью
- Вырожденный случай составного объекта, с одним листовым и множеством составных элементов

Composite

- Агрегирует объекты и представляет иерархию объектов в виде часть-целое

FACADE



ФАСАД

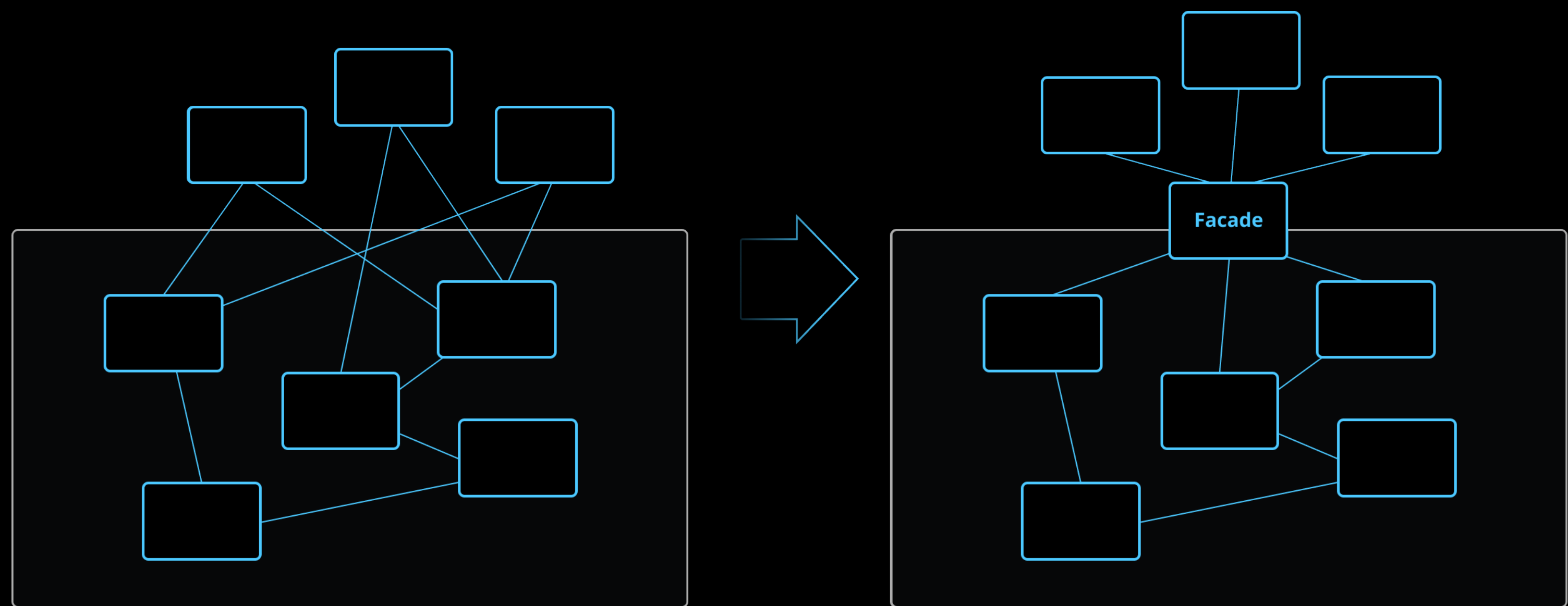
Предоставляет унифицированный
интерфейс вместо
набора интерфейсов
некоторой подсистемы

Определяет интерфейс более
высокого уровня,
который упрощает
использование подсистемы

Решаемая задача

- Предоставляет простой интерфейс к сложной подсистеме
- Устраняет большое число зависимостей между клиентами и классами подсистемы
- Может являться точкой входа для некоторого слоя

Паттерн не имеет чёткой реализации



Реализация
бизнес-логики

```
(new Facade())->someOperation();

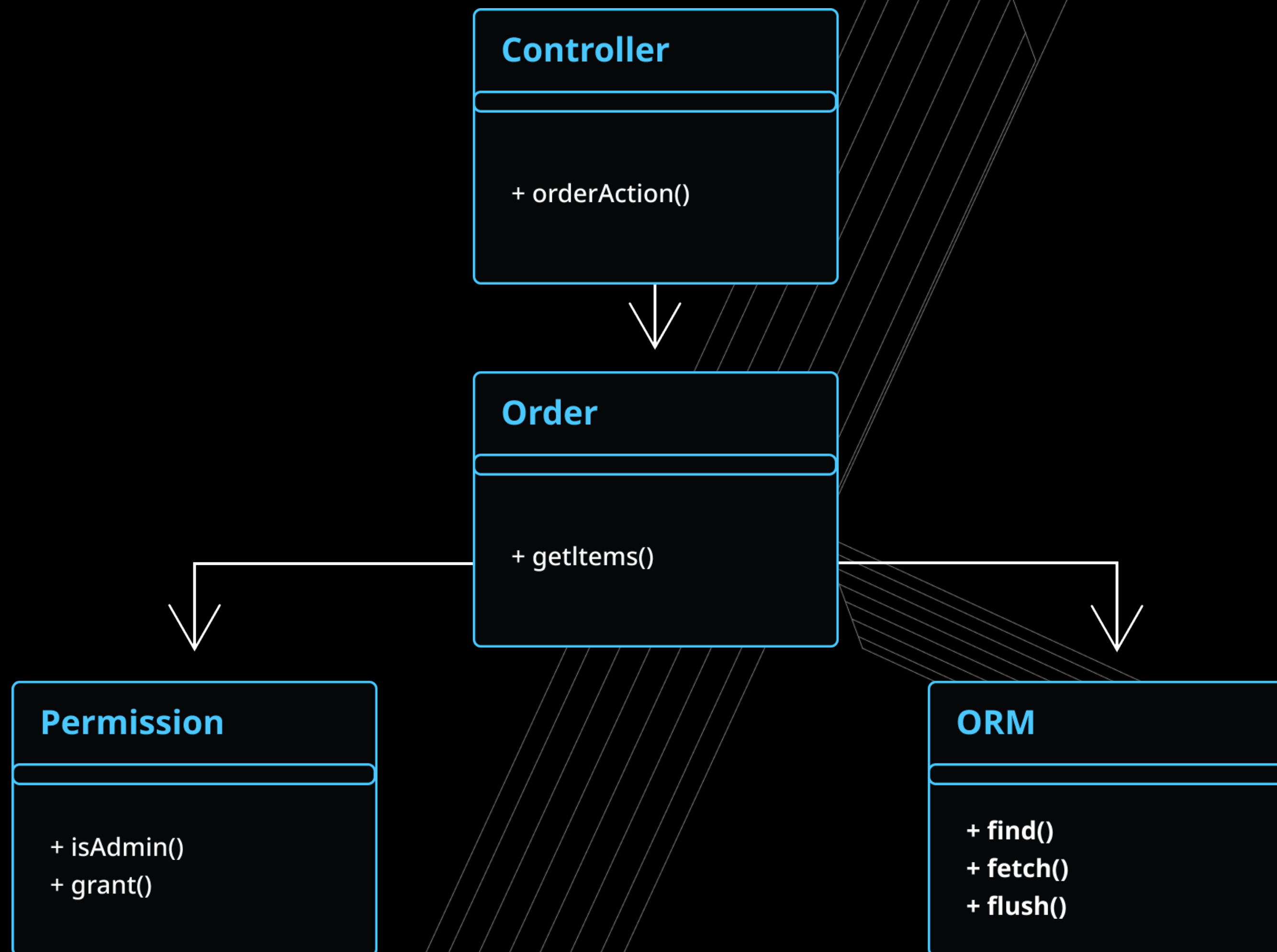
class Facade
{
    public function someOperation()
    {
        $first = (new First())->operation();
        $second = new Second();
        $second->method($first);
    }

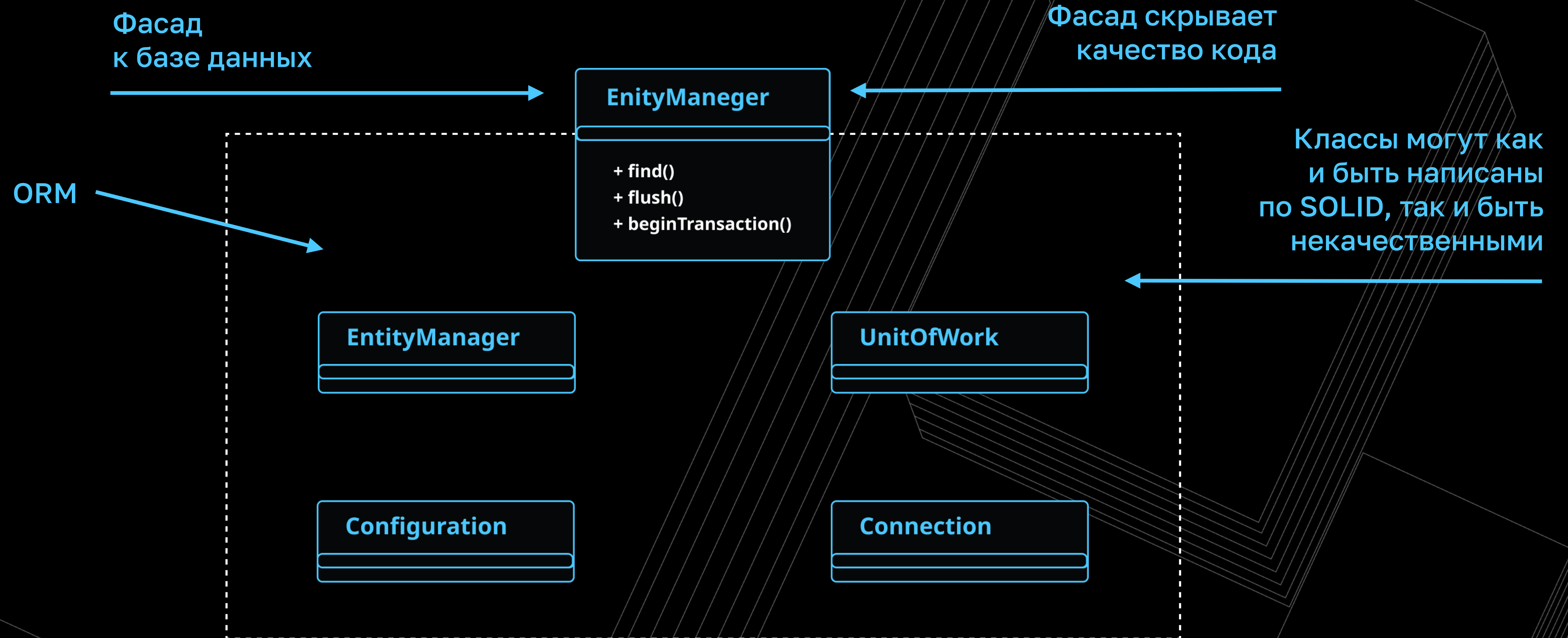
    // public function anotherSomeOperation() {}
}
```

Методы могут
инкапсулировать
свои задачи

Фасад
к правам доступа

Фасад
к базе данных





Плюсы

- Скрывает компоненты системы от внешних клиентов
- Разделяет систему на слои
- Удовлетворяет принципу слабой связанности (Low Coupling)
- Не мешает напрямую обращаться к подсистеме классов

Минусы



- Если Фасад содержит много бизнес-логики, он превращается в «божественный класс»

**Скрывает сложность
системы путём
сведения всех
возможных внешних
вызовов к одному
объекту**

avito.tech

Смотрите все серии на сайте
<https://avito.tech/patterns>

